

Backend Development

SOLID Principles & Authentication Types

PART 1 — SOLID Principles

S — Single Responsibility Principle (SRP)

A class or module should have one, and only one, reason to change.

- Each component does exactly one job.
- Easier to test, maintain, and debug.
- Example: Separate UserService (business logic) from UserRepository (data access).

O — Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

- Add new features by extending, not editing existing code.
- Reduces risk of breaking existing functionality.
- Example: Use strategy/plugin patterns instead of modifying core logic.

L — Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without altering program correctness.

- Child classes must honour the contract of the parent class.
- Prevents unexpected behaviour in polymorphic scenarios.
- Example: A Square class extending Rectangle must not break area calculations.

I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

- Prefer many small, focused interfaces over one large general-purpose one.
- Reduces coupling and unnecessary dependencies.
- Example: Split a fat IUserService into IAuthService, IProfileService, etc.

D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions.

- Depend on interfaces/abstractions, not concrete implementations.
- Enables easy swapping of dependencies (e.g., changing databases).
- Example: Inject a IEmailSender interface, not a concrete SmtplibEmailSender.

PART 2 — Authentication Types

1. Basic Authentication

Credentials (username:password) are Base64-encoded and sent in the HTTP Authorization header with every request.

- Simple to implement; no server-side session needed.
- Must always be used over HTTPS — Base64 is NOT encryption.
- Not suitable for production APIs; legacy systems only.

How it works: Client → Base64(user:pass) → Authorization: Basic → Server decodes & verifies.

2. Session-Based Authentication

After login, the server creates a session stored server-side (DB/Redis) and sends a session ID cookie to the client.

- Stateful — server must maintain session store.
- Easy to invalidate (just delete the session).
- Doesn't scale horizontally without a shared session store.

How it works: Login → Server stores session → Returns Set-Cookie: sessionId → Client sends cookie on each request.

3. Token-Based Authentication (JWT)

After login, the server issues a signed JSON Web Token containing claims (user ID, roles, expiry). The client stores and sends it with each request.

- Stateless — no server-side storage required.
- Scalable and great for microservices & SPAs.
- Tokens cannot be invalidated before expiry without a blocklist.

How it works: Login → Server signs JWT → Client stores in memory/cookie → Sends as Bearer token → Server verifies signature.

4. OAuth 2.0

An authorization framework allowing a third party to access resources on behalf of a user without sharing credentials.

- Separates authentication from authorisation.
- Supports multiple grant types: Authorization Code, Client Credentials, Implicit, Device.
- Industry standard for social login (Google, GitHub, etc.).

How it works: User → Auth Server → Authorization Code → Access Token + Refresh Token → Resource Server.

5. OpenID Connect (OIDC)

An identity layer built on top of OAuth 2.0 that adds authentication, returning an ID Token (JWT) with user identity info.

- Combines authentication (who you are) + authorisation (what you can do).
- Provides standardised /userinfo endpoint.
- Used by Google, Microsoft, Auth0, Okta.

How it works: OAuth 2.0 flow + ID Token (JWT) → Verified by client to authenticate the user.

6. API Key Authentication

A unique key is issued to a client application and sent with every API request, usually in a header or query parameter.

- Simple and stateless; ideal for server-to-server communication.
- No user identity — identifies the application, not a person.
- Keys should be rotated regularly and scoped to minimum permissions.

How it works: Client → X-API-Key: → Server validates against key store.

7. Multi-Factor Authentication (MFA)

Requires users to prove identity using two or more independent factors: something you know, have, or are.

- Dramatically reduces account compromise risk.
- Factors: Password (know) + OTP/TOTP (have) + Biometrics (are).
- Common implementations: TOTP (Google Authenticator), SMS OTP, hardware keys (FIDO2/WebAuthn).

How it works: Correct password → Prompt for 2nd factor (TOTP/SMS) → Validate → Grant access.

8. Biometric Authentication

Uses unique biological traits — fingerprint, face, iris — to verify identity, typically via device-level APIs.

- Very high security and user convenience.
- Often combined with FIDO2/WebAuthn for web apps.
- Biometric data never leaves the device; only a cryptographic key is sent.

How it works: Device captures biometric → Local match → Private key signs challenge → Server verifies signature.